Decoupling Deployment Boundaries from Memory Boundaries in Function Composition

Pranab Sarkar KubeFn Project (https://kubefn.com) ORCID: 0009-0009-8683-1481

March 2026

Abstract

The dominant abstraction in serverless and microservice composition conflates deployment isolation with data representation boundaries: each independently deployable unit occupies a separate address space, and every inter-unit data flow requires serialization, network transit, and deserialization. In large service graphs, this serialization tax consumes 30–40% of total CPU cycles and dominates end-to-end latency. The alternative—monolithic deployment—eliminates serialization but sacrifices independent deployability, the primary operational advantage of microservices.

This paper shows that deployment isolation and data representation boundaries can be decoupled, enabling independently deployable functions to execute over a shared in-memory object graph while preserving compositional semantics and practical isolation. This paper introduces **Memory-Continuous Architecture (MCA)**, a design pattern in which co-located functions share a single runtime process heap and communicate through direct object references rather than serialized byte streams. This paper presents the **HeapExchange**, a zero-copy shared object graph with versioned schema evolution, capacity governance, and causal audit logging. This work implements MCA across three language runtimes—JVM (classloader isolation, virtual threads, Netty), CPython (shared interpreter, `importlib` hot-loading), and Node.js (V8 isolate, `require`-based module loading)—demonstrating that the pattern is language-agnostic.

In full HTTP-cycle benchmarks against estimated microservice baselines, MCA achieves 3.8ms average latency for a 7-function JVM pipeline (4–18x improvement), 1.0ms for a 3-function Python ML inference pipeline (6–30x), and 0.3ms for a 3-function Node.js API gateway (20–100x). We discuss limitations including shared GC pauses, shared failure domains, and the absence of cross-runtime object sharing. KubeFn, the open-source reference implementation, integrates with Kubernetes through custom resource definitions and a reconciliation-loop operator.

---

1. Introduction

1.1 The Microservices Serialization Tax

The microservices architectural style [1, 2] decomposes applications into independently deployable services communicating over network protocols. This decomposition yields significant operational benefits: independent scaling, polyglot technology choices, fault isolation, and team autonomy. However, it introduces a structural cost that is rarely quantified at design time.

Every inter-service call requires the caller to serialize its request (typically to JSON or Protocol Buffers), transmit the bytes over a network socket, and have the receiver deserialize them back into in-memory objects. For a single hop, this overhead is modest—2 to 10 milliseconds for a typical intra-cluster HTTP call, depending on payload size and serialization format. But microservice graphs are rarely single-hop. A user-facing request to an e-commerce checkout system might traverse authentication, inventory, pricing, shipping calculation, tax computation, fraud scoring, and order assembly services. Seven hops at 2–10ms each produces 14–70ms of latency attributable purely to serialization and network transit, before any business logic executes.

Google's internal analysis of production microservice workloads found that serialization and deserialization consumed 30–40% of total CPU cycles across some service graphs [3]. This is not an implementation deficiency; it is a structural consequence of the architectural choice to place memory boundaries at every deployment boundary.

1.2 The Monolith/Microservices Tradeoff

The industry has long treated this as a binary choice. Monolithic deployment keeps all code in one process, enabling direct method calls with zero serialization overhead. But monoliths couple deployment: changing one function requires redeploying the entire application. This coupling slows release velocity, increases blast radius, and prevents independent scaling.

Microservices solve the deployment problem but create the serialization problem. Various mitigation strategies exist—service meshes reduce network overhead, binary serialization formats (Protocol Buffers, FlatBuffers, Cap'n Proto) reduce marshalling cost, sidecar proxies amortize connection management—but none eliminate the fundamental issue. Data must still cross a memory boundary at every service call.

1.3 Thesis

I argue that the conflation of deployment boundaries with memory boundaries is not a necessary property of independently deployable systems but rather an artifact of the container-per-service deployment model that became the default with the rise of Docker and Kubernetes. Specifically:

> *The dominant abstraction in serverless composition conflates deployment isolation with data representation boundaries. This paper shows these concerns can be decoupled, enabling independently deployable functions to execute over a shared in-memory object graph while preserving compositional semantics and practical isolation.*

This decoupling is achievable because the properties that developers actually require from "independent deployment"—independent versioning, independent release, independent rollback, per-function routing and observability—do not inherently require process-level isolation. They require *logical* isolation of code loading and lifecycle management, which can be provided within a single process through classloader hierarchies (JVM), module namespaces (Python), or module cache partitioning (Node.js).

1.4 Contribution

This paper makes the following contributions:

1. **Memory-Continuous Architecture (MCA)**, a formally described architectural pattern in which deployment boundaries are decoupled from memory boundaries, enabling independently deployable functions to share a process heap and communicate through direct object references.
2. **The HeapExchange**, a zero-copy shared object graph with typed capsules, versioned schema evolution, capacity governance (leak detection, stale eviction), and a causal audit log suitable for production use.
3. **A multi-runtime implementation** across three major language platforms (JVM, CPython, Node.js), demonstrating that MCA is not tied to any single language's memory model or module system.
4. **Quantitative evaluation** showing 4–100x latency improvements in full HTTP-cycle benchmarks across all three runtimes, with honest methodology, conservative speedup ranges, and explicit discussion of threats to validity.
5. **KubeFn**, an open-source, Kubernetes-native reference implementation with CRDs, a reconciliation-loop operator, and production-grade resilience primitives.

---

2. Background and Related Work

2.1 Traditional Architectures

Monoliths. A monolithic application runs as a single process. Function calls are method invocations on the same heap—zero serialization, zero network transit. The cost is operational: all code deploys together, scales together, and fails together.

Microservices. Fowler and Lewis [1] and Newman [2] describe microservices as independently deployable services organized around business capabilities. Each service owns its data and communicates through well-defined APIs. The cost is the serialization tax described in Section 1.1, plus the operational complexity of distributed systems: service discovery, load balancing, circuit breaking, distributed tracing, and eventual consistency.

Function-as-a-Service (FaaS). Serverless platforms (AWS Lambda, Google Cloud Functions, Azure Functions) take microservice decomposition to its logical extreme: individual functions. Most FaaS platforms run each function invocation in an isolated container or micro-VM, imposing cold start latency (100ms–10s for JVM

functions) in addition to serialization costs. Some platforms reuse warm containers, but the function-to-function communication path still traverses network boundaries.

2.2 Prior Shared-Runtime Approaches

OSGi (Open Service Gateway initiative). The OSGi specification [4] defines a module system for Java that uses classloaders to isolate bundles within a single JVM. OSGi provides a service registry through which bundles can discover and invoke each other. However, OSGi's complexity is well-documented: bundle resolution is NP-complete in the general case, the lifecycle model (installed, resolved, starting, active, stopping, uninstalled) creates intricate state machines, and the specification's treatment of class visibility across bundles produces the notorious `ClassNotFoundException` and `ClassCastException` failure modes that plagued Eclipse RCP and Apache ServiceMix deployments. Critically, OSGi was designed for modularity, not for zero-copy data sharing between independently deployable functions.

Java EE Application Servers. Enterprise Java application servers host multiple WAR and EAR deployments in a single JVM, sharing thread pools, connection pools, and JNDI resources. However, the communication model between deployed applications relies on remote EJB calls (network serialization), JMS messaging (serialization to byte streams), or shared databases (serialization to SQL). In-memory data sharing between deployments is neither supported nor safe under the Java EE specification.

Erlang/OTP. The BEAM virtual machine [5] runs millions of lightweight processes sharing a VM-level heap. Processes communicate through message passing, which in most implementations copies the message data into the receiving process's heap. While BEAM achieves remarkable fault isolation through its "let it crash" supervision model, the copy-on-send semantics mean that Erlang does not provide zero-copy data sharing across process boundaries. Furthermore, the BEAM ecosystem is limited to Erlang and Elixir.

GraalVM and Truffle. Oracle's GraalVM [6] supports polyglot execution—running JavaScript, Python, Ruby, and R on the JVM through the Truffle framework. GraalVM enables inter-language object sharing, but its primary design goal is polyglot interoperability rather than deployment-boundary decoupling. GraalVM native-image compilation eliminates JVM startup time but does not address the architectural question of how independently deployable units share memory.

Project Leyden. Project Leyden [7] aims to improve Java startup and warmup time through static images and pre-computed class initialization. While Leyden addresses the cold-start problem that plagues JVM serverless, it does not address inter-function serialization or deployment-boundary decoupling.

2.3 Serverless Systems with Shared State or Reduced Isolation

A growing body of work has explored relaxing the isolation constraints of serverless platforms to improve performance. We discuss the most closely related systems and differentiate MCA from each.

FAASM. Shillaker et al. [15] present Faasm, a serverless runtime that uses WebAssembly (Wasm) as the isolation mechanism, enabling functions to share memory regions within a single process. Faasm achieves lightweight isolation through Wasm's linear memory model and supports efficient state sharing via a two-tier state architecture (local and global). MCA differs from Faasm in three respects. First, MCA provides true zero-copy

object sharing—functions access the same heap-allocated object references with pointer identity preserved—whereas Faasm's shared memory regions require explicit serialization into a flat byte format compatible with Wasm's linear memory. Second, MCA supports multiple production language runtimes (JVM, CPython, Node.js) with their full standard libraries and ecosystems, whereas Faasm requires functions to be compiled to WebAssembly, which limits language support and precludes the use of native C-extension libraries (e.g., PyTorch, NumPy with BLAS backends). Third, MCA provides declarative schema evolution (HeapEnvelope with semantic versioning) and hot-swap with drain management, which Faasm does not address.

Nightcore. Jia and Witchel [16] present Nightcore, a serverless runtime optimized for microsecond-scale internal function calls through a managed threading model and optimized IPC. Nightcore achieves low latency by co-locating functions and replacing HTTP-based invocation with shared-memory message channels. However, Nightcore still serializes data across function boundaries—it optimizes the *transport* (replacing network sockets with shared-memory IPC) rather than eliminating the *representation boundary*. MCA eliminates both: functions share the same heap objects with no serialization whatsoever. Additionally, Nightcore does not support runtime hot-swap of individual functions or schema evolution of shared data.

Cloudburst. Sreekanti et al. [17] present Cloudburst, a stateful FaaS platform built on Anna, a low-latency key-value store with conflict-free replicated data types (CRDTs). Cloudburst provides a caching layer that keeps frequently accessed state local to function executors, reducing the latency of state access. However, Cloudburst's caching layer still serializes objects between the function runtime and the cache, and cross-function communication goes through the KVS layer with serialization at each boundary. MCA eliminates serialization entirely for co-located functions. Cloudburst's strength is distributed stateful computation with causal consistency guarantees, a use case that MCA does not address—MCA targets co-located function pipelines within a single process.

Boki. Jia and Witchel [18] present Boki, a serverless runtime that provides shared logs as a communication primitive, enabling functions to share state through append-only log streams with exactly-once semantics. Boki addresses the challenge of consistent state sharing across distributed function instances, which is orthogonal to MCA's focus. MCA targets the intra-process case: functions that can be co-located share heap objects with zero serialization. Boki targets the inter-process case: functions that must be distributed share state through replicated logs. The two approaches are complementary—a system could use MCA for co-located function pipelines and Boki-style shared logs for cross-node state coordination.

SAND. Akkus et al. [19] present SAND, a serverless platform that introduces application-level sandboxing, co-locating functions of the same application in a shared container with a local message bus. SAND reduces inter-function latency by avoiding network hops between co-located functions. MCA goes further: where SAND's local message bus still involves message serialization and copying, MCA's HeapExchange provides zero-copy object sharing with pointer identity. Additionally, MCA provides formal schema evolution, capacity governance, and multi-runtime support, which SAND does not address.

Summary of differentiation. The key property that distinguishes MCA from all of the above systems is *zero-copy object sharing without serialization*, where the consumer receives the identical object reference (same heap pointer) that the producer published. No existing system provides this property for independently deployable functions across multiple production language runtimes with declarative schema evolution and hot-swap support. Table 1 summarizes the comparison.

System	Isolation	Zero-Copy Objects	Multi-Runtime	Schema Evolution	Hot-Swap
FAASM [15]	WebAssembly	No (byte regions)	Wasm only	No	No
Nightcore [16]	Containers	No (shared-mem IPC)	Multi-language	No	No
Cloudburst [17]	Containers	No (KVS caching)	Python	No	No
Boki [18]	Containers	No (shared logs)	Multi-language	No	No
SAND [19]	Application sandbox	No (message bus)	Multi-language	No	No
MCA/KubeFn	Classloader/module	**Yes (heap refs)**	**JVM, Python, Node.js**	**Yes (HeapEnvelope)**	**Yes (drain + swap)**

2.4 Serialization Costs in Real Systems

The cost of serialization in microservice architectures is well-documented but often underestimated:

- Google engineers reported that serialization and deserialization of Protocol Buffers consumed 30–40% of CPU cycles in some internal microservice graphs [3]. This measurement included the CPU cost of marshalling and unmarshalling but excluded the network latency of transmitting the serialized bytes.
- Benchmarks of common serialization frameworks show that even efficient binary formats require 1–10 microseconds per object for serialization and a comparable amount for deserialization [8]. For a pipeline of N functions with M shared objects, the total serialization cost scales as O(N * M).
- A 2019 study of production microservice deployments at Alibaba found that network communication (including serialization) accounted for 50–70% of end-to-end latency in request paths traversing more than three services [9].

These measurements establish a clear cost model: every memory boundary in a microservice graph imposes a latency and CPU tax proportional to the number of objects crossing that boundary. Memory-Continuous Architecture eliminates this cost for functions within the same trust boundary.

---

3. Memory-Continuous Architecture

3.1 Core Principle: Deployment Boundary != Memory Boundary

The central insight of Memory-Continuous Architecture is that the deployment boundary—the unit of independent release, scaling, and lifecycle management—need not coincide with the memory boundary—the unit of address space isolation.

In traditional microservices, these boundaries are identical: each service runs in its own process (or container), with its own heap, and all inter-service communication crosses both a deployment boundary and a memory boundary. MCA separates these concerns:

- **Deployment boundary**: each function is an independently versionable, independently loadable unit of code. Functions can be added, removed, or updated without restarting the runtime.
- **Memory boundary**: functions within a trust group share a single process heap. Data flows between functions as direct object references—same pointer, same memory address, zero copies.

This separation preserves the operational benefits of microservices (independent deployment, per-function versioning, per-function scaling via routing weights) while eliminating the serialization tax for co-located functions.

3.2 The HeapExchange: Zero-Copy Shared Object Graph

The HeapExchange is the data plane of MCA. It is a concurrent, typed, versioned key-value store that lives in the shared process heap. Functions publish objects to the HeapExchange; other functions retrieve them by key. The critical property: **the consumer receives the same object reference that the producer published**. There is no serialization, no deserialization, no copying.

The HeapExchange API consists of three operations:

```
publish(key, value, type) -> HeapCapsule
get(key, type) -> Optional<value>
remove(key) -> boolean
```

Each published value is wrapped in a **HeapCapsule** that records provenance metadata:

- **key**: the lookup identifier
- **value**: the published object (stored by reference)
- **type**: the runtime type for type-safe retrieval
- **version**: a monotonically increasing version counter
- **publisherGroup**: which function group published the object
- **publisherFunction**: which specific function published it
- **publishedAt**: wall-clock timestamp of publication

This metadata enables the runtime to track data lineage, detect stale objects, and enforce access patterns.

3.3 Capsule Design and Schema Evolution

Shared mutable state is notoriously difficult to manage. The HeapExchange mitigates this through two mechanisms:

HeapEnvelope. When schema evolution is required, values are wrapped in a `HeapEnvelope<T>` that carries explicit version metadata:

```
HeapEnvelope(value, schemaKey, majorVersion, minorVersion,
             publishedAt, producerRevision, metadata)
```

The compatibility rule follows semantic versioning: consumers check `isCompatibleWith(expectedMajor)`. Same major version guarantees structural compatibility; minor version differences are tolerated as additive, backward-compatible changes.

SchemaVersion declaration. Functions declare which versions of shared objects they produce and consume via `SchemaVersion` records:

```
SchemaVersion(key, majorVersion, minorVersion,
              producerGroup, consumerGroups)
```

The runtime can validate at load time that no consumer expects a major version that no producer provides, catching schema incompatibilities before traffic is routed to the new function.

3.4 HeapGuard: Capacity Governance

A shared heap without governance is a memory leak waiting to happen. The HeapGuard enforces three invariants:

1. **Capacity limits**: the HeapExchange rejects `publish` calls when the object count exceeds a configurable maximum (default: 10,000 objects). This prevents runaway functions from exhausting the shared heap.
2. **Leak detection**: the guard tracks the last access time of each key. Objects that have not been accessed within a configurable staleness threshold are candidates for eviction.
3. **Stale eviction**: a periodic eviction sweep removes objects that exceed the staleness threshold, reclaiming heap space and preventing the accumulation of orphaned state.

The guard operates with minimal overhead: publish and access tracking use `ConcurrentHashMap` operations, and the eviction sweep runs on a background timer, not on the request hot path.

3.5 Function Loading and Classloader Isolation

MCA requires that independently deployable functions share a heap while maintaining code isolation. The mechanism is runtime-specific:

JVM: Classloader isolation. Each function group is loaded by a dedicated `FunctionGroupClassLoader` that implements a child-first delegation model. Platform classes (`java.*`, `javax.*`, `com.kubefn.api.*`, `org.slf4j.*`) delegate to the parent classloader, ensuring a single copy of the API and logging facade. Function classes use child-first resolution, enabling each group to carry its own dependencies without conflicts.

Discarding a `FunctionGroupClassLoader` unloads the entire function group, making the classes eligible for garbage collection. This enables hot-swap: load the new version's classloader, drain in-flight requests to the old version, discard the old classloader.

Python: `importlib` hot-loading. The Python runtime uses `importlib.import_module` and `importlib.reload` to load function modules into a shared CPython interpreter. All functions share the same Python object space. The HeapExchange is a dict-based store (`dict[str, HeapCapsule]`) protected by a reentrant lock for concurrent access.

Node.js: `require`-based module loading. The Node.js runtime loads function modules using `require()` within a single V8 isolate. All functions share the same JavaScript heap. The HeapExchange is a `Map`-based store. Module hot-swap uses `require.cache` invalidation.

3.6 Hot-Swap: Replacing Functions Without Restart

Traditional microservice deployments use rolling updates: start new pods, drain old pods, terminate. This works but imposes a deployment latency of seconds to minutes. MCA enables sub-second hot-swap within a running process:

1. **Load**: the new function version is loaded into a fresh classloader (JVM), imported into the interpreter (Python), or required into the module cache (Node.js).
2. **Drain**: the `DrainManager` stops routing new requests to the old version and waits for in-flight requests to complete (with a configurable timeout).
3. **Switch**: the router atomically updates its routing table to point to the new version.
4. **Unload**: the old classloader/module is discarded, and its classes become eligible for garbage collection.

The entire cycle completes in milliseconds for the common case (no in-flight requests) or seconds under load (waiting for drain).

3.7 Born-Warm: New Functions Inherit Warm Runtime State

Cold start is a persistent problem in serverless and FaaS platforms, particularly for JVM-based functions where JIT compilation, class loading, and connection pool initialization can take seconds. MCA eliminates cold start for co-located functions:

- **JIT warmth**: on the JVM, the shared process has already JIT-compiled hot paths in the Netty event loop, Jackson serialization, and the HeapExchange itself. New functions benefit from these compiled code paths immediately.
- **Connection pools**: database connections, HTTP client pools, and cache clients managed by the runtime are shared. A newly loaded function does not need to establish its own connections.
- **Heap state**: the HeapExchange already contains objects published by other functions. A new function can begin consuming shared state on its first invocation.

This is the “born-warm” property: new functions are productive from their first invocation because they inherit the warm runtime state of the shared process.

3.8 Revision-Scoped State

When multiple revisions of a function group coexist during a canary deployment, the runtime must ensure request-level consistency. Each request is pinned to a specific revision through a `RevisionContext` that is set at dispatch time and propagated via thread-local storage:

```
RevisionContext(requestId, Map<groupName, revisionId>, createdAt)
```

The `RevisionManager` supports weighted traffic splitting: during a canary deployment, 90% of requests can be routed to the stable revision and 10% to the canary, with the weight adjustable at runtime without redeployment. Both revisions execute against the same HeapExchange, enabling zero-cost A/B comparison of function behavior.

3.9 The Trust Model

MCA requires a trust boundary: functions sharing a heap must trust each other not to corrupt shared state, exhaust shared resources, or behave maliciously. This is the same trust model as a monolithic application—code within the process boundary is trusted.

The trust boundary in KubeFn is the **function group**: a set of functions that are co-deployed, co-scaled, and share a classloader and HeapExchange. Functions in different groups are isolated by classloader boundaries (JVM) or separate runtime instances. Cross-group communication uses the traditional microservice path (HTTP/gRPC) with full serialization.

This is not a security weakness; it is an explicit architectural choice. MCA is appropriate for functions that would otherwise be methods in the same monolithic application. It is not appropriate for multi-tenant isolation or zero-trust environments.

---

4. Multi-Runtime Implementation

4.1 JVM Runtime

The JVM runtime is the most feature-complete implementation, leveraging the JVM's mature concurrency and class-loading infrastructure.

Classloaders. `FunctionGroupClassLoader` extends `URLClassLoader` with child-first delegation. Platform API classes (`com.kubefn.api.*`) are loaded from the parent to ensure type identity across function groups. Function classes are loaded from child URLs (directories or JARs containing compiled `.class` files).

Virtual threads. Request dispatch uses Java 21 virtual threads [10] via `Executors.newVirtualThreadPerTaskExecutor()`. User function code never executes on Netty's event loop threads, preventing blocking operations from stalling the I/O pipeline. The `FnGraphEngine` uses `StructuredTaskScope.ShutdownOnFailure` for parallel pipeline steps, ensuring clean cancellation on failure.

Netty server. The HTTP server is built on Netty 4.x, providing non-blocking I/O with minimal memory allocation. Request bodies are extracted from Netty's `ByteBuf` and passed to functions as byte arrays; response bodies are wrapped in `Unpooled.wrappedBuffer` to avoid copying. The Netty pipeline handles HTTP codec, request aggregation, and dispatch.

HeapExchange. The JVM HeapExchange uses `ConcurrentHashMap<String, HeapCapsule<?>>` as its backing store, providing O(1) lock-free reads and segment-locked writes. Version counters use `AtomicLong` for contention-free incrementing. Thread-local context (`ThreadLocal<String>`) tracks the current function's identity for publisher attribution without synchronization on the hot path.

Pipeline composition. The `FnGraphEngine` implements the `FnPipeline` interface, enabling declarative function composition:

```
pipeline.step(AuthFunction.class)
        .parallel(InventoryFunction.class, PricingFunction.class)
        .step(TaxFunction.class)
        .step(AssemblyFunction.class)
        .build()
        .execute(request);
```

All steps execute as in-process method calls. Parallel steps use structured concurrency on virtual threads. The entire pipeline shares a single HeapExchange instance, enabling zero-copy data flow between steps.

4.2 Python Runtime

The Python runtime adapts MCA to CPython's single-interpreter model.

Shared interpreter. All functions run in the same CPython interpreter, sharing the same global `id()` space. When Function A publishes a NumPy array to the HeapExchange, Function B receives the same array object—same `id()`, same underlying memory buffer. For ML inference pipelines, this eliminates the cost of serializing and deserializing large tensors.

Dict-based HeapExchange. The Python HeapExchange uses a `dict[str, HeapCapsule]` protected by a `threading.RLock`. Read operations (`get`) do not acquire the lock in the common case (dict reads are atomic in CPython due to the GIL), reducing contention. Write operations (`publish`, `remove`) acquire the lock for compound atomic updates (version increment + store + audit).

Module loading. Functions are loaded as Python modules via `importlib.import_module`. Hot-swap uses `importlib.reload` after invalidating the module in `sys.modules`. The ASGI server (`uvicorn` or equivalent) continues serving requests during reload.

GIL implications. CPython's Global Interpreter Lock serializes Python bytecode execution. For CPU-bound functions, this limits parallelism to one core. However, for I/O-bound functions (the common case for API handlers and ML inference with C-extension backends like NumPy, PyTorch, and TensorFlow), the GIL is released during I/O and C-extension calls. The HeapExchange benefits from the GIL: dict operations are inherently thread-safe, eliminating the need for fine-grained locking.

4.3 Node.js Runtime

The Node.js runtime adapts MCA to V8's single-threaded event loop.

V8 isolate. All functions run in the same V8 isolate, sharing the same JavaScript heap. Object references are direct—`===` identity is preserved across function boundaries. The HeapExchange is a `Map` instance (O(1) lookups, insertion-order iteration).

Module loading. Functions are loaded via `require()`. Hot-swap invalidates `require.cache` entries and re-requires the module. The event loop continues processing requests during reload.

Event loop implications. Node.js is single-threaded. All function executions are interleaved on the event loop. This means the HeapExchange requires no synchronization—there are no concurrent mutations. This is both a strength (zero locking overhead) and a limitation (CPU-bound functions block the event loop). For the typical use case of API gateways, rate limiters, and request routers, the single-threaded model is ideal.

4.4 Common Patterns Across Runtimes

Despite the significant differences between the JVM, CPython, and V8, the MCA implementation follows a common pattern across all three:

Concern	JVM	Python	Node.js
Isolation	Classloader	Module/namespace	`require.cache`
HeapExchange store	`ConcurrentHashMap`	`dict`	`Map`
Concurrency model	Virtual threads	GIL + async	Event loop
Zero-copy mechanism	Object reference	Object reference	Object reference
Hot-swap	Classloader discard	`importlib.reload`	Cache invalidation
Audit log	`HeapAuditLog` class	`list[dict]`	`Array`

The API surface is identical: `publish(key, value, type)`, `get(key)`, `remove(key)`. The zero-copy guarantee holds uniformly: the consumer receives the same object reference the producer published.

---

5. System Architecture

5.1 Function Model

A KubeFn function is characterized by:

- **Deploy unit**: compiled `.class` files (JVM), `.py` modules (Python), or `.js` modules (Node.js). The runtime provides the framework dependencies (Jackson, Netty, Caffeine, SLF4J for JVM; aiohttp for Python; express/fastify for Node.js).

- **Routing**: each function registers one or more HTTP routes (method + path pattern). The `FunctionRouter` resolves incoming requests to function handlers with O(1) lookup for exact routes and O(N) prefix matching for wildcard routes.
- **Lifecycle**: functions implement a `KubeFnHandler` interface with a single `handle(KubeFnRequest) -> KubeFnResponse` method. The runtime manages initialization, health checking, and shutdown.

5.2 HeapExchange Details

The HeapExchange in the JVM implementation integrates several subsystems:

Capsule metadata. Every stored object is wrapped in a `HeapCapsule<T>` record containing the key, value, type, version (monotonically increasing `AtomicLong`), publisher group, publisher function, and publication timestamp. The type parameter enables type-safe retrieval: `get("pricing:result", PricingResult.class)` returns `Optional<PricingResult>` and logs a type-mismatch warning if the stored type is incompatible.

Audit log. The `HeapAuditLog` records every mutation (publish, access, remove) with the acting function's identity and the current revision context. This log is queryable through the admin API and feeds into the causal introspection engine.

Capacity governance. The `HeapGuard` enforces a maximum object count, tracks per-key access timestamps, and provides a `findStaleKeys()` method for periodic eviction. The guard's `checkPublish` method is called on every `publish` invocation and returns an error string if the operation should be blocked, keeping the guard logic out of the critical path's success case.

Metrics. The HeapExchange exposes publish count, get count, hit count, miss count, and hit rate as a `HeapMetrics` record. These metrics are exported to the `KubeFnMetrics` singleton for Prometheus-compatible scraping.

5.3 Resilience

Shared-runtime execution concentrates risk: a misbehaving function can affect all co-located functions. KubeFn addresses this through four resilience primitives:

Circuit breakers. Each function has a dedicated `CircuitBreaker` (Resilience4j [11]) configured with a 50% failure rate threshold, a 10-call sliding window, and a 30-second open-state duration. When a function's circuit breaker trips, the runtime returns 503 Service Unavailable and routes to a registered fallback function if one exists. Circuit breaker state transitions are logged and exposed through the admin API.

Drain manager. The `DrainManager` tracks in-flight request counts per function group using atomic counters. During hot-swap, the drain manager stops accepting new requests for the group being swapped and waits (with a configurable timeout) for in-flight requests to complete. This prevents request loss during deployment.

Request timeouts. Every function invocation is wrapped in a `CompletableFuture.get(timeout, TimeUnit.MILLISECONDS)` call. If a function exceeds its configured timeout (default: 30 seconds), the request is cancelled, the circuit breaker records a failure, and the runtime returns 504 Gateway Timeout.

Concurrency limits. Per-group concurrency is bounded by a `Semaphore` with a configurable maximum (default: 100 concurrent requests per group). This prevents one function group from monopolizing the virtual thread pool.

Fallback registry. The `FallbackRegistry` maps function identifiers to fallback handlers that execute when the primary function's circuit breaker is open or when the primary function throws an exception. Fallbacks run in the same process with access to the same HeapExchange, enabling graceful degradation (e.g., returning cached results).

5.4 Observability

MCA introduces a unique observability opportunity: because all function invocations occur within a single process, the runtime can observe causal relationships between function calls and heap mutations with nanosecond precision, without the coordination overhead of distributed tracing.

Causal Capture Engine. The `CausalCaptureEngine` sits on the hot path and captures structured events (request start/end, function start/end, heap publish/get/remove, circuit breaker trips, pipeline lifecycle, drain events) into a lock-free ring buffer (`CausalEventRing`). Events carry a monotonically increasing event ID, nanosecond timestamp, request ID, and type-specific metadata.

The ring buffer uses pre-allocated storage and atomic operations for zero-contention append. The engine can be toggled at runtime (enabled/disabled) through the admin API without restarting the process.

Request Trace Assembly. The `RequestTraceAssembler` reconstructs complete request traces from the event stream, correlating function invocations with heap mutations. Each `RequestTrace` includes the entry function, total duration, per-step breakdown, heap mutations caused by the request, and any errors encountered.

Trace search. The engine supports filtered trace search across multiple dimensions: function group, function name, minimum duration, and error presence. This enables targeted debugging ("show me all requests to the pricing function that took more than 10ms and had errors").

OpenTelemetry integration. The `KubeFnTracer` wraps OpenTelemetry [12] spans around every function invocation, exporting trace data to standard backends (Jaeger, Zipkin, OTLP). Each span includes the function group, function name, revision ID, request ID, and duration as attributes.

Prometheus metrics. The `KubeFnMetrics` singleton records invocation counts, latency histograms, circuit breaker trips, timeouts, heap publish/get operations, and hit rates. Metrics are exposed on a Prometheus-compatible `/metrics` endpoint.

5.5 Kubernetes Integration

KubeFn integrates with Kubernetes through two custom resource definitions and a reconciliation-loop operator:

KubeFnGroup CRD. Defines a function group: its name, runtime type (JVM, Python, Node.js), resource requests/limits, replica count, and configuration. The operator reconciles `KubeFnGroup` resources into `Deployment`, `Service`, and `ConfigMap` Kubernetes objects.

KubeFnFunction CRD. Defines an individual function within a group: its handler class, route mappings, timeout, concurrency limit, and circuit breaker configuration. The operator watches `KubeFnFunction` resources and triggers hot-swap in the running pod when a function is added, updated, or removed.

Operator. The KubeFn operator is built on the Java Operator SDK (JOSDK) [13] and Fabric8 Kubernetes client. It runs a standard reconciliation loop: watch CRD events, compare desired state with actual state, and apply the minimal set of Kubernetes API calls to converge.

Helm chart. A Helm chart packages the operator, CRDs, RBAC roles, and example function groups for single-command installation: `helm install kubefn kubefn/kubefn`.

---

6. Evaluation

This work evaluates MCA's latency characteristics across all three runtime implementations using realistic multi-function pipelines. All benchmarks measure full HTTP request-response cycles, including network transit, request parsing, routing, function execution, heap operations, and response serialization. I compare against estimated equivalent microservice latencies based on published intra-cluster HTTP overhead measurements.

6.1 Methodology

Benchmarking tool. I use `hey` [14], a widely-used HTTP load generator. All benchmarks run 1,000 requests with 10 concurrent connections from the same machine to eliminate network variability. Results are averaged over 5 independent runs.

Environment. All benchmarks run on a single machine (Apple M-series, 16GB RAM) to isolate runtime performance from network topology. KubeFn runtimes run as standalone processes (not in Kubernetes) to eliminate orchestrator overhead from the measurement.

Microservice baseline. We estimate equivalent microservice latency as N * H, where N is the number of functions in the pipeline and H is the per-hop latency. I use two baseline ranges: - **Intra-pod (localhost)**: 2ms per hop (optimistic: same-node, loopback interface) - **Cross-node**: 5–10ms per hop (realistic: intra-cluster with service mesh)

These baselines are conservative; production microservice hops frequently exceed 10ms due to serialization, service mesh overhead, retries, and connection establishment. However, I note explicitly that **these are estimated baselines, not measured deployments of equivalent microservice systems**. We did not implement the same business logic as separate microservices and benchmark them end-to-end. The speedup ratios should therefore be interpreted as estimates of the improvement attributable to eliminating serialization and network hops, not as measured head-to-head comparisons.

What I measure. Full HTTP cycle: the `hey` client sends an HTTP request to the KubeFn runtime, which routes it through the multi-function pipeline, and returns the response. The measurement includes Netty request parsing, router resolution, function dispatch, HeapExchange operations, response serialization, and Netty response writing.

What we do not measure. Cold start time, JIT warmup (benchmarks run after a 100-request warmup), or Kubernetes orchestration latency. These costs are orthogonal to MCA's value proposition.

6.2 JVM Benchmark: Checkout Pipeline

Setup. A 7-function e-commerce checkout pipeline: AuthFunction (validates token), InventoryFunction (checks stock), PricingFunction (calculates price), ShippingFunction (computes shipping cost), TaxFunction (applies tax rules), FraudFunction (scores transaction risk), and AssemblyFunction (combines results into final order). Functions communicate through the HeapExchange: each function publishes its result and reads predecessors' results via direct object reference.

Results.

Metric	Value
Average latency	3.8 ms
p50 latency	2.5 ms
p95 latency	4.4 ms
p99 latency	7.1 ms
Throughput	2,550 req/s

Comparison. Equivalent microservices: 7 hops * 2ms (optimistic) = 14ms; 7 * 10ms (cross-node) = 70ms. KubeFn achieves **4x improvement** over optimistic localhost microservices and **18x improvement** over realistic cross-node microservices. The speedup range of **4–18x** is conservative: it does not account for JSON serialization/deserialization of the shared objects at each hop, which would add 1–5ms per hop in the microservice case.

Analysis. The 3.8ms average includes Netty HTTP parsing (~0.2ms), virtual thread dispatch (~0.1ms), 7 function invocations with HeapExchange reads and writes (~2.5ms total), and Netty response writing (~0.2ms). The remaining ~0.8ms is attributable to Jackson serialization of the final response and overhead from the resilience stack (circuit breaker permission checks, drain manager accounting, causal event capture).

6.3 Python Benchmark: ML Inference Pipeline

Setup. A 3-function ML inference pipeline: FeatureFunction (extracts features from input), PredictFunction (runs model inference), ExplainFunction (generates prediction explanation). The feature vector, model output, and explanation are shared through the HeapExchange as Python objects (dicts and NumPy arrays).

Results.

Metric	Value
Average latency	1.0 ms

Metric	Value
p50 latency	0.6 ms
p95 latency	0.9 ms
p99 latency	1.8 ms
Throughput	7,455 req/s

Comparison. Equivalent microservices: 3 hops * 2ms = 6ms (optimistic); 3 * 10ms = 30ms (cross-node). KubeFn achieves **6–30x improvement**. The speedup is particularly significant for ML pipelines because the alternative involves serializing NumPy arrays or tensors at each service boundary—a cost of 1–50ms depending on array size—which MCA eliminates entirely.

Analysis. The GIL limits CPU-bound parallelism, but for this pipeline (I/O-bound input parsing, C-extension inference, dict-based explanation assembly), the GIL is released during the computationally intensive phases. The HeapExchange dict operations are atomic under the GIL without explicit locking.

6.4 Node.js Benchmark: API Gateway Pipeline

Setup. A 3-function API gateway: RateLimitFunction (token bucket check), AuthFunction (JWT validation), RouteFunction (upstream routing and response assembly). Functions share rate-limit state and auth context through the HeapExchange.

Results.

Metric	Value
Average latency	0.3 ms
p50 latency	0.2 ms
p95 latency	0.5 ms
p99 latency	0.8 ms
Throughput	33,085 req/s

Comparison. Equivalent microservices: 3 hops * 2ms = 6ms (optimistic); 3 * 10ms = 30ms (cross-node). KubeFn achieves **20–100x improvement**. Node.js achieves the highest speedup ratio because its single-threaded event loop has the lowest per-invocation overhead (no thread scheduling, no lock acquisition, no context switching).

Analysis. The 0.3ms average includes HTTP parsing (~0.05ms), three function invocations with Map lookups (~0.15ms total), and response writing (~0.1ms). The single-threaded model means zero synchronization overhead for HeapExchange operations. The limitation is CPU-bound functions: a function that blocks the event loop for 10ms delays all other in-flight requests.

6.5 Hot-Swap Evaluation

This work evaluatesd hot-swap under load by continuously sending requests (10 concurrent connections) while replacing a function in the JVM checkout pipeline. During a 200-request demo run, the drain manager successfully held in-flight requests while the new classloader was loaded, and the router atomically switched to the new version with zero dropped requests. Average swap time was under 50ms with no in-flight requests and under 500ms under load (waiting for drain).

Caveat. This evaluation was conducted with a small number of requests (200) on a single machine. I have not yet validated hot-swap behavior under production-scale load (thousands of concurrent connections, sustained throughput). The drain timeout, classloader GC timing, and interaction with JIT deoptimization under sustained load remain to be characterized. We consider production-scale hot-swap evaluation necessary future work.

6.6 Threats to Validity

Estimated baselines, not measured comparisons. The most significant threat to the evaluation is that the microservice baselines are estimated from published per-hop latency data, not measured from deployed equivalent microservice systems running the same business logic. A proper comparison would require implementing the same function pipelines as independent microservices, deploying them on the same hardware (or equivalent cloud instances), and benchmarking end-to-end. I report speedup as a range (e.g., 4–18x) to partially account for this uncertainty, but the true speedup for any specific workload depends on factors we did not measure: payload size, serialization format, service mesh configuration, and network conditions.

Single-machine benchmarks. All benchmarks run on a single machine, which eliminates network variability but does not capture production network conditions, container scheduling jitter, or NUMA effects on multi-socket servers.

Synthetic workloads. The benchmark functions perform lightweight computation (token validation, dict lookups, simple arithmetic). Production functions with heavy CPU usage (e.g., image processing, large data transformations) would show a smaller speedup ratio because the serialization tax becomes a smaller fraction of total latency. The reported speedups are most representative of I/O-bound and coordination-heavy workloads.

Warmup effects. Benchmarks run after a warmup period. The JVM's JIT compiler has optimized hot paths before measurement begins. Cold-start performance is not measured; MCA's born-warm property addresses cold-start but is not quantified here.

GC impact not isolated. The JVM benchmarks do not isolate garbage collection pauses from function execution latency. The p99 latency of 7.1ms for the JVM pipeline (versus a p50 of 2.5ms) suggests GC-induced tail latency, but we did not instrument GC logs to confirm this. A shared heap amplifies GC impact because all co-located functions contribute to allocation pressure and all are paused during stop-the-world collections. We discuss this further in Section 7.

---

7. Discussion

7.1 When to Use MCA vs. Microservices

MCA is not a replacement for microservices. It is a complementary pattern for a specific architectural context: multiple functions that are maintained by the same team, share trust boundaries, and compose tightly in request processing.

Use MCA when: - Functions form a pipeline (A calls B calls C) and share data at each step. - The serialization tax dominates latency (many hops, large shared objects). - Functions are maintained by the same team and share a deployment lifecycle. - Cold-start latency is unacceptable (e.g., JVM-based serverless).

Use microservices when: - Functions are maintained by different teams with different release cadences. - Functions require different trust boundaries or security isolation. - Functions need independent technology stacks (different languages, databases, frameworks). - Functions have vastly different scaling profiles (one function needs 100 replicas, another needs 2).

In practice, a large system will use both patterns: MCA for tightly-coupled function pipelines within a bounded context, and microservices for communication between bounded contexts.

7.2 Trust Boundaries and Security Implications

MCA functions share a process heap. A malicious or buggy function can read or corrupt any object in the HeapExchange, exhaust heap memory, or crash the process. This is the same risk profile as a monolithic application.

The mitigation is organizational: function groups should correspond to trust boundaries. Functions within a group are authored by the same team, reviewed through the same process, and deployed through the same pipeline. Cross-group communication uses standard microservice protocols with full serialization and network isolation.

For environments requiring stronger isolation (multi-tenant platforms, regulatory compliance), MCA is inappropriate. The trust model is explicit: shared heap implies shared trust.

7.3 Garbage Collection Impact

A shared JVM heap means shared garbage collection. When the GC performs a stop-the-world pause, *all* co-located functions are paused simultaneously, regardless of which function generated the garbage. This is a meaningful concern for latency-sensitive workloads.

The severity depends on the GC algorithm. With the G1 collector (JDK default), mixed GC pauses of 10–50ms are common under high allocation rates. With ZGC (available since JDK 15, production-ready since JDK 21), pause times are bounded to sub-millisecond regardless of heap size, because ZGC performs concurrent relocation. KubeFn recommends ZGC for production deployments (`-XX:+UseZGC`), and the benchmarks use ZGC.

However, even with ZGC, a function that allocates aggressively can increase GC overhead (concurrent GC cycles consume CPU) and degrade throughput for all co-located functions. The HeapGuard's capacity limits provide a coarse-grained mitigation: by bounding the number of objects in the HeapExchange, the guard limits the

contribution of shared state to GC pressure. Per-function allocation tracking and throttling is an area for future work.

7.4 Failure Domains

The most significant operational risk of MCA is that functions sharing a JVM share a failure domain. If one function triggers an `OutOfMemoryError`, the entire JVM process terminates, taking all co-located functions with it. Similarly, a function that enters an infinite loop (consuming a virtual thread indefinitely) does not directly crash the process but consumes resources that may starve other functions.

KubeFn mitigates this through several mechanisms: - **HeapGuard capacity limits** prevent any single function from publishing unbounded objects to the HeapExchange. - **Request timeouts** kill long-running function invocations after a configurable deadline. - **Concurrency limits** (semaphore-based) prevent one function from consuming all available virtual threads. - **Circuit breakers** stop routing traffic to failing functions, allowing the healthy functions to continue serving.

These mitigations reduce the *probability* of cascading failures but do not eliminate the *possibility*. A function that allocates large objects on the stack or local variables (outside the HeapExchange) can still trigger OOM without tripping the HeapGuard. In practice, the risk is comparable to running multiple WAR files in a Java EE application server—a well-understood operational model with known failure modes.

For workloads requiring hard isolation, the correct architectural choice is separate function groups (separate JVM processes), with cross-group communication via HTTP.

7.5 Comparison to GraalVM Native Image and Project Leyden

GraalVM native-image [6] compiles Java applications ahead-of-time into native executables, eliminating JVM startup time and reducing memory footprint. Project Leyden [7] takes a similar approach within the OpenJDK ecosystem. Both address the cold-start problem but do not address the serialization-boundary problem.

MCA and native-image/Leyden are complementary: a KubeFn runtime compiled with GraalVM native-image would combine sub-100ms startup with zero-copy function composition. The main obstacle is that native-image's closed-world assumption conflicts with MCA's dynamic classloader-based function loading. A native-image-compatible MCA implementation would require ahead-of-time registration of all function classes, sacrificing runtime hot-swap.

---

8. Limitations and Future Work

We organize the limitations of the current system into categories and discuss corresponding future work directions.

8.1 Shared Failure Domain

As discussed in Section 7.4, the shared JVM model means a shared failure domain. One function's OOM kills all co-located functions. While HeapGuard limits and concurrency controls mitigate this, they cannot prevent all failure modes. Future work includes investigating OS-level memory cgroups per function group within a single process (leveraging Linux cgroup v2 memory controllers) and exploring cooperative memory budgeting where functions declare and the runtime enforces per-function allocation budgets.

8.2 Python GIL Limits True Parallelism

CPython's Global Interpreter Lock prevents true parallel execution of Python bytecode. For CPU-bound function pipelines, the GIL serializes execution, limiting throughput to a single core. The mitigation is to use C-extension libraries (NumPy, PyTorch) that release the GIL during computation, or to run multiple Python runtime processes with a load balancer. CPython 3.13 introduced an experimental free-threaded mode (PEP 703) that removes the GIL. Future work includes evaluating MCA on free-threaded CPython, which could enable true parallel function execution while retaining the shared-heap property.

8.3 No Cross-Runtime HeapExchange

The current implementation requires all functions in a group to use the same language runtime. A JVM function and a Python function cannot share HeapExchange objects because their object representations are incompatible (a Java `HashMap` and a Python `dict` have different memory layouts). This means that polyglot pipelines—where, for example, a Java authentication function feeds into a Python ML inference function—must use serialization at the language boundary, losing the zero-copy property.

Future work includes investigating shared-memory regions with a common columnar format (similar to Apache Arrow) as an intermediate representation, or leveraging GraalVM Truffle's polyglot object protocol for JVM-hosted languages. Neither approach preserves true zero-copy semantics with native object identity, so this remains an open research challenge.

8.4 Schema Evolution is Declarative but Not Enforced

The HeapEnvelope and SchemaVersion mechanisms provide declarative schema compatibility checking: functions declare which schema versions they produce and consume, and the runtime can validate compatibility at load time. However, the current implementation does not enforce schema contracts at runtime. A function can publish any object under any key, bypassing the schema declaration. Furthermore, there is no schema registry, no automated migration path for breaking schema changes, and no compile-time tooling to verify that a function's code matches its declared schema.

Future work includes a schema registry integrated with the Kubernetes operator (schema declarations as CRD annotations), compile-time annotation processors that verify schema compatibility, and runtime enforcement that rejects publishes that violate declared schemas.

8.5 No Formal Verification of Isolation Properties

We claim that classloader isolation provides practical isolation between function groups: each group has its own namespace, its own dependency versions, and its classloader can be discarded independently. However, I have not formally verified these isolation properties. Known edge cases include: - Static fields in classes loaded by the parent classloader (e.g., `java.lang.System` properties) are shared across all function groups. - JNI libraries loaded via `System.loadLibrary` are process-global and cannot be loaded by multiple classloaders. - `ThreadLocal` values set by one function group persist on carrier threads and may leak to other groups if not cleaned up.

Formal verification of classloader isolation boundaries, possibly using a model checker or through a type-system-level proof, is future work.

8.6 Production Deployment Data

All evaluation data in this paper comes from single-machine benchmarks with synthetic workloads. We do not yet have data from production deployments handling real user traffic. Production environments introduce factors absent from the benchmarks: variable payload sizes, bursty traffic patterns, interactions with Kubernetes scheduling and resource limits, GC behavior under sustained multi-hour load, and classloader leak accumulation over days of hot-swap cycles.

We consider production deployment validation essential for establishing MCA's practical viability and plan to pursue it through early-adopter partnerships.

8.7 Cross-Process and Cross-Node HeapExchange

For function groups that exceed single-process capacity, a shared-memory HeapExchange (using `MappedByteBuffer` on the JVM or `mmap` on POSIX systems) could extend the zero-copy property across processes on the same node, with a binary serialization fallback for cross-node communication.

8.8 Deterministic Replay

The CausalCaptureEngine records all function invocations and heap mutations with nanosecond precision. A natural extension is deterministic replay: given a captured trace, re-execute the exact sequence of function calls with the exact heap state. This would enable time-travel debugging for production incidents.

8.9 Autonomous Optimization

The runtime has complete visibility into function call graphs, heap access patterns, and latency profiles. Future versions could automatically fuse frequently co-invoked functions (eliminating dispatch overhead), memoize pure-function subgraphs (eliminating redundant computation), and pre-warm heap state based on predicted request patterns.

---

9. Conclusion

Memory-Continuous Architecture addresses a structural inefficiency in microservice systems: the conflation of deployment boundaries with memory boundaries. I have shown that these concerns can be decoupled, enabling independently deployable functions to execute over a shared in-memory object graph while preserving compositional semantics—independent versioning, independent release, per-function routing, and per-function observability.

The HeapExchange provides a typed, versioned, governed zero-copy data plane with capacity limits, leak detection, stale eviction, and causal audit logging. The multi-runtime implementation across JVM, CPython, and Node.js demonstrates that MCA is a language-agnostic architectural pattern, not a JVM-specific optimization.

Benchmarks show 4–18x latency improvement for JVM pipelines, 6–30x for Python ML inference, and 20–100x for Node.js API gateways, measured as full HTTP request-response cycles. These improvements come from eliminating serialization, deserialization, and network transit at every function boundary. We acknowledge that these comparisons use estimated microservice baselines and that production validation remains future work.

MCA is not a replacement for microservices. It is a third option between monoliths and microservices, applicable when functions share trust boundaries and compose tightly in request processing. The trust model is explicit: shared heap implies shared trust, shared garbage collection, and shared failure domain. Cross-boundary communication continues to use standard microservice protocols.

Compared to recent work on optimized serverless runtimes—FAASM [15], Nightcore [16], Cloudburst [17], Boki [18], and SAND [19]—MCA's distinguishing contribution is zero-copy object sharing without serialization across independently deployable functions, combined with multi-runtime support, declarative schema evolution, and hot-swap with drain management. I have been candid about the limitations: shared GC pauses, shared failure domains, no cross-runtime object sharing, and the absence of production deployment data. These limitations define the research agenda for future work.

KubeFn, the open-source reference implementation, is available at https://kubefn.com and https://github.com/kubefn/kubefn. It integrates with Kubernetes through custom resource definitions, supports hot-swap deployment without restart, and includes production-grade resilience primitives (circuit breakers, drain management, timeouts, fallbacks) and observability (causal introspection, OpenTelemetry tracing, Prometheus metrics).

References

[1] J. Lewis and M. Fowler, "Microservices: A definition of this new architectural term," martinfowler.com, Mar. 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html

[2] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.

[3] S. Kanev, J. P. Darago, K. Hazelwood, P. Ranganathan, B. Moseley, G.-Y. Wei, and D. Brooks, "Profiling a warehouse-scale computer," in *Proc. ACM/IEEE 42nd International Symposium on Computer Architecture (ISCA)*, 2015, pp. 158–169. doi: 10.1145/2749469.2750392

[4] OSGi Alliance, “OSGi Core Release 8 Specification,” 2020. [Online]. Available: https://docs.osgi.org/specification/

[5] J. Armstrong, *Programming Erlang: Software for a Concurrent World*, 2nd ed. Pragmatic Bookshelf, 2013.

[6] T. Wuerthinger et al., “GraalVM: Run Programs Faster Anywhere,” Oracle, 2019. [Online]. Available: https://www.graalvm.org/

[7] M. Reinhold, “Project Leyden: Beginnings,” OpenJDK, May 2022. [Online]. Available: https://openjdk.org/projects/leyden/notes/01-beginnings

[8] A. Sumaray and S. K. Makki, “A comparison of data serialization formats for optimal efficiency and interoperability of programmatic interfaces,” in *Proc. International Conference on Information and Knowledge Engineering (IKE)*, 2012, pp. 243–249.

[9] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, D. Liu, Q. Xiang, and C. He, “Latency-based SLA-aware autoscaling for cloud microservices,” in *Proc. IEEE International Conference on Cloud Computing (CLOUD)*, 2019, pp. 35–42.

[10] A. Bateman, R. Pressler, et al., “JEP 444: Virtual Threads,” OpenJDK, 2023. [Online]. Available: https://openjdk.org/jeps/444

[11] R. Vos, “Resilience4j: Fault tolerance library designed for Java 8 and functional programming,” 2023. [Online]. Available: https://resilience4j.readme.io/

[12] OpenTelemetry Authors, “OpenTelemetry Specification,” Cloud Native Computing Foundation, 2024. [Online]. Available: https://opentelemetry.io/docs/specs/otel/

[13] Container Solutions, “Java Operator SDK,” 2024. [Online]. Available: https://javaoperatorsdk.io/

[14] R. Saito, “hey: HTTP load generator,” 2023. [Online]. Available: https://github.com/rakyll/hey

[15] S. Shillaker and P. Pietzuch, “Faasm: Lightweight Isolation for Efficient Stateful Serverless Computing,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2020, pp. 419–433.

[16] Z. Jia and E. Witchel, “Nightcore: Efficient and Scalable Serverless Computing for Latency-Sensitive, Interactive Microservices,” in *Proc. 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021, pp. 152–166. doi: 10.1145/3445814.3446701

[17] V. Sreekanti, C. Wu, X. C. Lin, J. Schleier-Smith, J. E. Gonzalez, J. M. Hellerstein, and A. Tumanov, “Cloudburst: Stateful Functions-as-a-Service,” in *Proc. VLDB Endowment*, vol. 13, no. 11, 2020, pp. 2438–2452. doi: 10.14778/3407790.3407836

[18] Z. Jia and E. Witchel, “Boki: Stateful Serverless Computing with Shared Logs,” in *Proc. 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021, pp. 691–707. doi: 10.1145/3477132.3483541

[19] I. E. Akkus, R. Chen, I. Rimac, M. Stein, K. Satzke, A. Beck, P. Aditya, and V. Hilt, “SAND: Towards High-Performance Serverless Computing,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2018, pp. 923–935.